
ADO.NET y Entity Framework Core con SQLite

Manual completo de acceso a datos con C# 14 y .NET 10: SQL directo, abstracciones ADO.NET, CRUD, transacciones, migraciones, relaciones y criterios para elegir

Finalidad

Pensado para alumnado que conoce los fundamentos de C# y quiere comprender el acceso a bases de datos desde la capa más cercana a SQL hasta un ORM moderno, utilizando exclusivamente SQLite como motor de persistencia.

Al terminar este manual podrás:

- Comprender qué es **ADO.NET**, qué responsabilidades asume y cuándo conviene utilizar SQL directo.
- Conectar C# 14 y .NET 10 con SQLite mediante **Microsoft.Data.Sqlite**.
- Trabajar con **DbConnection**, **DbCommand**, parámetros y **DbDataReader**.
- Implementar un CRUD completo, transacciones, tratamiento de errores y consultas mediante las API asíncronas.
- Dominar pooling, timeout, aislamiento, CommandType, DataTable, metadatos y proveedores.
- Explicar qué problema resuelve un **ORM** y qué costes introduce.
- Configurar **Entity Framework Core 10** con SQLite, modelos, DbContext y migraciones.
- Implementar CRUD, consultas LINQ y relaciones uno a muchos, uno a uno y muchos a muchos.
- Elegir con criterio entre ADO.NET, EF Core o un enfoque híbrido.

Edición didáctica 2026 · .NET 10

Índice

1. Mapa del manual y proyecto conductor	6
Estructura orientativa	6
2. Bases de datos relacionales y SQLite	7
Qué aporta SQLite	7
Sistema de tipos dinámico	7
3. Qué es ADO.NET y cuándo utilizarlo	8
Costes del SQL directo	8
4. Preparar C# 14 y .NET 10	9
Modelo sencillo	9
Actividades propuestas	9
5. Diseñar el esquema SQLite	10
Decisiones importantes	10
6. Cadenas de conexión y rutas fiables	11
7. DbConnection: abrir, usar y cerrar	12
Regla de ciclo de vida	12
8. Abstracciones y proveedor SQLite	13
9. DbCommand y sus tres ejecuciones principales	14
CommandText, CommandTimeout y CommandType	14
10. Parámetros y protección frente a inyección SQL	15
Tipos, null y tamaño	15
11. DbDataReader: leer filas de forma eficiente	16
Nulos y accesorios tipados	16
12. Inicializar la base con SQL directo	17
13. CRUD ADO.NET: crear	18

14. CRUD ADO.NET: consultar	19
Detalle	19
Listado paginado con orden determinista	19
15. CRUD ADO.NET: actualizar y eliminar	20
Actualizar	20
Eliminar	20
16. Encapsular un CRUD completo	21
Responsabilidades	21
17. Transacciones ADO.NET	22
18. API asíncrona y la realidad de SQLite	23
Qué hacer	23
19. Errores, locking y timeout	24
Política de reintento	24
20. Pooling y ciclo de conexiones	25
21. WAL y concurrencia en SQLite	26
22. Aislamiento, transacciones diferidas y savepoints	27
23. CommandType y capacidades del proveedor	28
24. DataTable y DataSet	29
Límite del proveedor SQLite	29
25. Metadatos, PRAGMA y esquema	30
26. Rendimiento con SQL directo	31
Índices y plan	31
BLOB incremental	31
27. Probar código ADO.NET	32
Qué comprobar	32

28. Qué es un ORM y qué problema resuelve	33
Qué no resuelve	33
29. Instalar y configurar EF Core 10 con SQLite	34
Aplicación de consola	34
ASP.NET Core	34
30. Crear entidades	35
31. DbContext y configuración Fluent	36
Convenciones, atributos y Fluent API	36
32. Migraciones: evolucionar el esquema	37
Buenas prácticas	37
33. CRUD EF Core: crear y consultar	38
Crear	38
Consultar	38
34. CRUD EF Core: actualizar y eliminar	39
Actualizar entidad seguida	39
Eliminar entidad seguida	39
Operaciones directas	39
35. Seguimiento de cambios y estados	40
36. LINQ y traducción a SQL	41
IQueryable frente a IEnumerable	41
Inspección	41
37. Relación uno a muchos	42
38. Uno a uno y muchos a muchos	43
Uno a uno	43
Muchos a muchos	43
39. Cargar relaciones	44

40. Transacciones y concurrencia en EF Core	45
Token de concurrencia gestionado por la aplicación	45
41. SQL directo e interoperabilidad con EF Core	46
Consultas SQL parametrizadas	46
Compartir conexión y transacción	46
42. Limitaciones de EF Core con SQLite	47
43. Rendimiento con EF Core	48
Consultas compiladas	48
44. Probar EF Core con SQLite	49
Matriz mínima	49
45. Elegir ADO.NET, EF Core o ambos	50
Criterio práctico	50
46. Proyecto integrador: Biblioteca de aula	51
Parte A: ADO.NET	51
Parte B: EF Core	51
Entrega	51
47. Diagnóstico de errores frecuentes	52
Secuencia de diagnóstico	52
48. Checklist final	53
ADO.NET	53
EF Core	53
49. Glosario	54
50. Referencias	55

1. Mapa del manual y proyecto conductor

El proyecto conductor es una biblioteca con autores, libros, etiquetas y préstamos. Primero construiremos la persistencia con SQL directo; después mantendremos el mismo modelo mediante EF Core. La comparación será concreta y verificable.

Bloque	Tecnología	Resultado
Fundamentos	SQLite y modelo relacional	tablas, claves, tipos y restricciones
Acceso directo	ADO.NET y Microsoft.Data.Sqlite	conexión, comandos, lectores y CRUD
Fiabilidad	transacciones y errores	atomicidad, timeout, locking y reintentos
Propiedades avanzadas	ADO.NET abstracto	pooling, aislamiento, DataTable y proveedores
ORM	Entity Framework Core 10	modelo, DbContext, migraciones y LINQ
Relaciones	EF Core y SQLite	navegaciones, claves foráneas y carga
Decisión	comparación aplicada	criterios para elegir o combinar

Estructura orientativa

Carpetas del proyecto	
1	BibliotecaDatos/
2	├─ Models/
3	├─ Ado/
4	│ └─ ConexionSqlite.cs
5	│ └─ LibroRepository.cs
6	├─ Ef/
7	│ └─ BibliotecaContext.cs
8	├─ Migrations/
9	└─ Program.cs
10	└─ biblioteca.db

Objetivo didáctico

El propósito no es memorizar dos sintaxis. Es comprender qué ocurre entre una operación de C# y el fichero SQLite, qué trabajo realiza cada capa y dónde deben vivir las reglas, las consultas y las transacciones.

2. Bases de datos relacionales y SQLite

Una base de datos relacional organiza información en tablas. Cada fila representa una ocurrencia; cada columna, un atributo. Las claves primarias identifican filas y las claves foráneas relacionan tablas. Restricciones e índices protegen integridad y aceleran consultas.

Qué aporta SQLite

- Motor relacional embebido: no necesita instalar ni administrar un servidor independiente.
- La base de datos suele residir en un único fichero transportable.
- Soporta transacciones ACID, claves foráneas, índices, vistas, triggers y gran parte de SQL.
- Resulta apropiado para enseñanza, escritorio, móvil, pruebas, cachés locales y servicios con escritura moderada.
- No es la opción natural para muchas escrituras concurrentes, clústeres o administración centralizada compleja.

Sistema de tipos dinámico

Clase SQLite	Valores habituales	Tipos .NET frecuentes
INTEGER	enteros de 1 a 8 bytes	int, long, bool
REAL	número IEEE 754	double, float
TEXT	texto UTF-8 o UTF-16	string, fechas serializadas, Guid
BLOB	secuencia de bytes	byte[]
NULL	ausencia de valor	null o DBNull.Value

Longitud y precisión

SQLite acepta declaraciones como VARCHAR(100) o DECIMAL(10,2), pero no aplica por sí mismo esas facetas como otros motores. Utiliza CHECK, validación de aplicación y tipos adecuados cuando sean parte de la regla.

3. Qué es ADO.NET y cuándo utilizarlo

ADO.NET es el conjunto de abstracciones de .NET para conectarse a fuentes de datos, ejecutar comandos, leer resultados y controlar transacciones. No es un ORM: no mantiene automáticamente un grafo de objetos ni genera todo el SQL a partir de LINQ.

Necesidad	Por qué encaja ADO.NET
Aprender SQL y acceso a datos	hace visibles conexiones, parámetros, filas y transacciones
Consulta muy específica	control total sobre SQL, columnas y plan
Ruta de alto rendimiento medida	evita seguimiento y materialización innecesarios
Aplicación pequeña	puede evitar incorporar un ORM completo
Interoperabilidad	permite trabajar con las abstracciones DbConnection y DbCommand

Costes del SQL directo

- Hay que escribir y mantener SQL, parámetros y mapeo de cada fila.
- El desarrollador controla manualmente el ciclo de conexión y transacción.
- Los cambios de esquema exigen una estrategia propia o una herramienta externa.
- Las relaciones no se convierten automáticamente en navegaciones entre objetos.
- La repetición puede crecer si no se encapsulan operaciones comunes.

No es una tecnología antigua

ADO.NET sigue siendo la base de los proveedores modernos. Microsoft.Data.Sqlite implementa sus abstracciones y el proveedor SQLite de EF Core se construye sobre esa biblioteca.

4. Preparar C# 14 y .NET 10

Terminal

```
1 dotnet new console -n BibliotecaDatos
2 cd BibliotecaDatos
3 dotnet add package Microsoft.Data.Sqlite --version 10.0.0
4 dotnet build
5 dotnet run
```

BibliotecaDatos.csproj

```
1 <Project Sdk="Microsoft.NET.Sdk">
2   <PropertyGroup>
3     <OutputType>Exe</OutputType>
4     <TargetFramework>net10.0</TargetFramework>
5     <LangVersion>14.0</LangVersion>
6     <ImplicitUsings>enable</ImplicitUsings>
7     <Nullable>enable</Nullable>
8   </PropertyGroup>
9 </Project>
```

El SDK de .NET 10 utiliza C# 14 de forma predeterminada. Declarar LangVersion puede resultar útil en material didáctico, aunque normalmente basta con TargetFramework net10.0. Mantén alineadas las versiones principales de los paquetes de datos.

Modelo sencillo

Models/Libro.cs

```
1 public sealed record Libro(
2     long Id,
3     string Titulo,
4     string Isbn,
5     int Anio,
6     long AutorId);
```

Actividades propuestas

1. Crea el proyecto net10.0, instala Microsoft.Data.Sqlite y comprueba las referencias con dotnet list package.
2. Activa Nullable y explica por qué una columna opcional debe reflejarse en el tipo de C#.

5. Diseñar el esquema SQLite

schema.sql

```
1 PRAGMA foreign_keys = ON;
2
3 CREATE TABLE IF NOT EXISTS Autores (
4     Id INTEGER PRIMARY KEY AUTOINCREMENT,
5     Nombre TEXT NOT NULL,
6     Pais TEXT NULL
7 );
8
9 CREATE TABLE IF NOT EXISTS Libros (
10    Id INTEGER PRIMARY KEY AUTOINCREMENT,
11    Titulo TEXT NOT NULL CHECK (length(Titulo) BETWEEN 2 AND 160),
12    Isbn TEXT NOT NULL UNIQUE,
13    Anio INTEGER NOT NULL CHECK (Anio BETWEEN 1450 AND 2100),
14    AutorId INTEGER NOT NULL,
15    FOREIGN KEY (AutorId) REFERENCES Autores(Id) ON DELETE RESTRICT
16 );
17
18 CREATE INDEX IF NOT EXISTS IX_Libros_AutorId ON Libros(AutorId);
19 CREATE INDEX IF NOT EXISTS IX_Libros_Titulo ON Libros(Titulo);
```

Decisiones importantes

- INTEGER PRIMARY KEY se asocia al rowid de SQLite; AUTOINCREMENT añade reglas más estrictas y no siempre es necesario.
- NOT NULL expresa obligatoriedad en la base de datos, no solo en C#.
- UNIQUE impide ISBN duplicados incluso si dos procesos validan a la vez.
- CHECK respalda rangos y longitudes críticas.
- La clave foránea necesita estar habilitada y debe acompañarse de una política de borrado consciente.
- Los índices de claves foráneas y filtros frecuentes evitan recorridos completos.

6. Cadenas de conexión y rutas fiables

CSHARP

```
1 string ruta = Path.Combine(
2     AppContext.BaseDirectory,
3     "datos",
4     "biblioteca.db");
5
6 Directory.CreateDirectory(Path.GetDirectoryName(ruta)!);
7
8 var builder = new SqliteConnectionStringBuilder
9 {
10     DataSource = ruta,
11     Mode = SqliteOpenMode.ReadWriteCreate,
12     Pooling = true,
13     DefaultTimeout = 30,
14     ForeignKeys = true
15 };
16
17 string cadenaConexion = builder.ToString();
```

Una ruta relativa se interpreta respecto al directorio de trabajo, que puede cambiar al ejecutar desde el editor, una prueba o un servicio. `AppContext.BaseDirectory` ofrece una base estable, aunque en aplicaciones publicadas suele convenir una carpeta de datos configurable y con permisos de escritura.

Propiedad	Uso
Data Source	ruta del fichero, :memory: o nombre de memoria compartida
Mode	crear, lectura-escritura, solo lectura o memoria
Pooling	reutilizar conexiones nativas; activado de forma predeterminada
Default Timeout	tiempo de reintento ante bloqueos para comandos implícitos
Foreign Keys	ejecutar PRAGMA foreign_keys al abrir
Cache	caché privada o compartida; cambia el comportamiento de bloqueo
Vfs	seleccionar el sistema virtual de ficheros; disponible en la versión 10

No concatenes rutas ni claves

`SqliteConnectionStringBuilder` construye y escapa la cadena. El campo `Password` solo funciona si la biblioteca nativa de SQLite utilizada incorpora cifrado; no cifra una base estándar por sí solo.

7. DbConnection: abrir, usar y cerrar

CSHARP

```
1 using System.Data.Common;
2 using Microsoft.Data.Sqlite;
3
4 DbConnection conexion = new SqliteConnection(cadenaConexion);
5
6 try
7 {
8     conexion.Open();
9     Console.WriteLine($"Estado: {conexion.State}");
10    Console.WriteLine($"Base de datos: {conexion.Database}");
11 }
12 finally
13 {
14     conexion.Dispose();
15 }
```

La forma habitual es **using** o **await using**, que garantiza Dispose incluso cuando ocurre una excepción. Cerrar pronto no perjudica al pooling: se devuelve la conexión interna al pool para reutilizarla.

CSHARP

```
1 await using DbConnection conexion = new SqliteConnection(cadenaConexion);
2 await conexion.OpenAsync();
3
4 // Crear y ejecutar comandos dentro de este alcance.
```

Regla de ciclo de vida

- Crea y abre una conexión por unidad de trabajo; no mantengas una conexión global.
- No compartas `SqliteConnection`, `SqliteCommand` ni `SqliteDataReader` entre hilos.
- Termina el lector antes de reutilizar la conexión para operaciones incompatibles.
- Una base `:memory:` desaparece al cerrar su última conexión; las pruebas deben conservar una abierta.

8. Abstracciones y proveedor SQLite

System.Data.Common define clases base independientes del proveedor. Microsoft.Data.Sqlite aporta SQLiteConnection, SQLiteCommand, SQLiteDataReader, SQLiteParameter, SQLiteTransaction y SQLiteFactory.

Abstracción	Implementación SQLite	Responsabilidad
DbConnection	SQLiteConnection	sesión lógica con la base
DbCommand	SQLiteCommand	SQL, parámetros, timeout y transacción
DbParameter	SQLiteParameter	valor tratado como dato
DbDataReader	SQLiteDataReader	lectura secuencial de filas
DbTransaction	SQLiteTransaction	unidad atómica de trabajo
DbProviderFactory	SQLiteFactory	creación abstracta de objetos

CSHARP

```
1  DbProviderFactory fabrica = SQLiteFactory.Instance;
2
3  await using DbConnection conexion = fabrica.CreateConnection()
4      ?? throw new InvalidOperationException("No se pudo crear la conexión.");
5
6  conexion.ConnectionString = cadenaConexion;
7  await conexion.OpenAsync();
8
9  await using DbCommand comando = conexion.CreateCommand();
10 comando.CommandText = "SELECT COUNT(*) FROM Libros";
11 long total = (long)(await comando.ExecuteScalarAsync() ?? 0L);
```

Usar las clases base facilita sustituir el proveedor, pero SQL, tipos y capacidades siguen siendo específicos del motor. La portabilidad nunca es automática solo por declarar DbConnection.

9. DbCommand y sus tres ejecuciones principales

Método	Resultado	Uso
ExecuteNonQuery	número de filas afectadas	CREATE, INSERT, UPDATE y DELETE
ExecuteScalar	primera columna de la primera fila	COUNT, MAX o RETURNING escalar
ExecuteReader	DbDataReader	cero, una o muchas filas y columnas

CSHARP

```
1  await using DbCommand comando = conexion.CreateCommand();
2  comando.CommandText = """
3      SELECT COUNT(*)
4      FROM Libros
5      WHERE Anio >= $anio
6      """;
7
8  DbParameter parametro = comando.CreateParameter();
9  parametro.ParameterName = "$anio";
10 parametro.Value = 2000;
11 comando.Parameters.Add(parametro);
12
13 long cantidad = (long)(comando.ExecuteScalar() ?? 0L);
```

CommandText, CommandTimeout y CommandType

CommandText contiene el SQL. CommandTimeout controla cuánto tiempo se reintenta ante bloqueos; 0 significa sin límite. SQLite no admite procedimientos almacenados, por lo que Microsoft.Data.Sqlite trabaja con **CommandType.Text** y no ofrece parámetros de salida.

10. Parámetros y protección frente a inyección SQL

CSHARP

```
1  await using DbCommand comando = conexion.CreateCommand();
2  comando.CommandText = """
3      SELECT Id, Titulo, Isbn, Anio, AutorId
4      FROM Libros
5      WHERE Titulo LIKE $busqueda ESCAPE '\'
6      ORDER BY Titulo
7      LIMIT $limite
8      """;
9
10 DbParameter busqueda = comando.CreateParameter();
11 busqueda.ParameterName = "$busqueda";
12 busqueda.Value = $"%{texto}%";
13 comando.Parameters.Add(busqueda);
14
15 comando.Parameters.Add(new SqliteParameter("$limite", SqliteType.Integer)
16 {
17     Value = 50
18 });
```

Un parámetro separa el texto SQL del valor. Nunca intentes proteger una consulta escapando manualmente comillas. Los parámetros pueden ocupar posiciones donde SQLite espera literales, pero no sustituyen nombres de tabla, columna ni palabras clave.

Tipos, null y tamaño

CSHARP

```
1  var pais = comando.CreateParameter();
2  pais.ParameterName = "$pais";
3  pais.Value = autor.Pais is null ? DBNull.Value : autor.Pais;
4  comando.Parameters.Add(pais);
5
6  var titulo = new SqliteParameter("$titulo", SqliteType.Text)
7  {
8      Value = libro.Titulo,
9      Size = 160
10 };
11 comando.Parameters.Add(titulo);
```

- Microsoft.Data.Sqlite admite prefijos \$, @ y :: usa una convención y respétala.
- DBNull.Value representa SQL NULL; Size puede truncar TEXT o BLOB, pero no sustituye la validación.
- SqliteType permite elegir el tipo de almacenamiento; valida aparte todo fragmento que no pueda parametrizarse.

11. DbDataReader: leer filas de forma eficiente

DbDataReader expone un flujo no almacenado de resultados. Read avanza a la siguiente fila. Es eficiente porque no carga toda la consulta en memoria, pero la conexión y el comando deben seguir vivos mientras se lee.

CSHARP

```
1  await using DbCommand comando = conexion.CreateCommand();
2  comando.CommandText = """
3      SELECT Id, Titulo, Isbn, Anio, AutorId
4      FROM Libros
5      ORDER BY Titulo
6      """;
7
8  await using DbDataReader lector = comando.ExecuteReader();
9
10 int ordId = lector.GetOrdinal("Id");
11 int ordTitulo = lector.GetOrdinal("Titulo");
12 int ordIsbn = lector.GetOrdinal("Isbn");
13 int ordAnio = lector.GetOrdinal("Anio");
14 int ordAutorId = lector.GetOrdinal("AutorId");
15
16 while (lector.Read())
17 {
18     var libro = new Libro(
19         lector.GetInt64(ordId),
20         lector.GetString(ordTitulo),
21         lector.GetString(ordIsbn),
22         lector.GetInt32(ordAnio),
23         lector.GetInt64(ordAutorId));
24     Console.WriteLine(libro.Titulo);
25 }
```

Nulos y accesores tipados

CSHARP

```
1  string? pais = lector.IsDBNull(lector.GetOrdinal("Pais"))
2      ? null
3      : lector.GetString(lector.GetOrdinal("Pais"));
```

- Combina accesores tipados con ordinales resueltos una sola vez para evitar conversiones y búsquedas repetidas.
- Comprueba IsDBNull antes de leer una columna opcional; GetSchemaTable expone los metadatos del resultado.
- CommandBehavior.SequentialAccess resulta útil para BLOB grandes y lectura secuencial.

12. Inicializar la base con SQL directo

Ado/BaseDatos.cs

```
1 public static void Inicializar(string cadenaConexion)
2 {
3     using DbConnection conexion = new SqliteConnection(cadenaConexion);
4     conexion.Open();
5
6     using DbCommand comando = conexion.CreateCommand();
7     comando.CommandText = """
8         PRAGMA foreign_keys = ON;
9
10        CREATE TABLE IF NOT EXISTS Autores (
11            Id INTEGER PRIMARY KEY,
12            Nombre TEXT NOT NULL,
13            Pais TEXT NULL
14        );
15
16        CREATE TABLE IF NOT EXISTS Libros (
17            Id INTEGER PRIMARY KEY,
18            Titulo TEXT NOT NULL,
19            Isbn TEXT NOT NULL UNIQUE,
20            Anio INTEGER NOT NULL,
21            AutorId INTEGER NOT NULL REFERENCES Autores(Id)
22        );
23        """;
24     comando.ExecuteNonQuery();
25 }
```

CREATE TABLE IF NOT EXISTS resulta práctico para una demostración, pero no reemplaza un sistema de evolución del esquema. Cuando la aplicación cambia versiones hay que registrar qué cambios se aplicaron, preservar datos y probar actualizaciones.

Una sola fuente de verdad

No mantengas a la vez un schema.sql manual y migraciones de EF Core modificando la misma base sin una política. Decide quién es responsable del esquema.

13. CRUD ADO.NET: crear

LibroRepository.cs

```
1 public long Crear(LibroCrear entrada)
2 {
3     using DbConnection conexion = CrearConexion();
4     conexion.Open();
5
6     using DbCommand comando = conexion.CreateCommand();
7     comando.CommandText = """
8         INSERT INTO Libros (Titulo, Isbn, Anio, AutorId)
9         VALUES ($titulo, $isbn, $anio, $autorId)
10        RETURNING Id;
11        """;
12
13     Agregar(comando, "$titulo", entrada.Titulo);
14     Agregar(comando, "$isbn", entrada.Isbn);
15     Agregar(comando, "$anio", entrada.Anio);
16     Agregar(comando, "$autorId", entrada.AutorId);
17
18     return (long)(comando.ExecuteScalar()
19         ?? throw new DataException("No se devolvió el identificador."));
20 }
```

CSHARP

```
1 private static void Agregar(DbCommand comando, string nombre, object? valor)
2 {
3     DbParameter parametro = comando.CreateParameter();
4     parametro.ParameterName = nombre;
5     parametro.Value = valor ?? DBNull.Value;
6     comando.Parameters.Add(parametro);
7 }
```

RETURNING permite recuperar el identificador en el mismo comando. En código compatible con versiones antiguas puede usarse `last_insert_rowid()` en la misma conexión, pero no debe consultarse desde otra.

14. CRUD ADO.NET: consultar

Detalle

CSHARP

```
1 public Libro? ObtenerPorId(long id)
2 {
3     using DbConnection conexion = CrearConexion();
4     conexion.Open();
5
6     using DbCommand comando = conexion.CreateCommand();
7     comando.CommandText = ""
8         SELECT Id, Titulo, Isbn, Anio, AutorId
9         FROM Libros
10        WHERE Id = $id
11        ""
12     Agregar(comando, "$id", id);
13
14     using DbDataReader lector = comando.ExecuteReader();
15     return lector.Read() ? MapearLibro(lector) : null;
16 }
```

Listado paginado con orden determinista

CSHARP

```
1 public List<Libro> ObtenerTodos(int pagina, int tamano)
2 {
3     using DbConnection conexion = CrearConexion();
4     conexion.Open();
5
6     using DbCommand comando = conexion.CreateCommand();
7     comando.CommandText = ""
8         SELECT Id, Titulo, Isbn, Anio, AutorId
9         FROM Libros
10        ORDER BY Titulo, Id -- orden estable para LIMIT/OFFSET
11        LIMIT $tamano OFFSET $salto
12        ""
13     Agregar(comando, "$tamano", tamano);
14     Agregar(comando, "$salto", (pagina - 1) * tamano);
15
16     using DbDataReader lector = comando.ExecuteReader();
17     var libros = new List<Libro>();
18     while (lector.Read()) libros.Add(MapearLibro(lector));
19     return libros;
20 }
```

15. CRUD ADO.NET: actualizar y eliminar

Actualizar

CSHARP

```
1 public bool Actualizar(long id, LibroEditar entrada)
2 {
3     using DbConnection conexion = CrearConexion();
4     conexion.Open();
5
6     using DbCommand comando = conexion.CreateCommand();
7     comando.CommandText = ""
8         UPDATE Libros
9         SET Titulo = $titulo,
10            Isbn = $isbn,
11            Anio = $anio,
12            AutorId = $autorId
13         WHERE Id = $id
14     "";
15     Agregar(comando, "$id", id);
16     Agregar(comando, "$titulo", entrada.Titulo);
17     Agregar(comando, "$isbn", entrada.Isbn);
18     Agregar(comando, "$anio", entrada.Anio);
19     Agregar(comando, "$autorId", entrada.AutorId);
20     return comando.ExecuteNonQuery() == 1;
21 }
```

Eliminar

CSHARP

```
1 public bool Eliminar(long id)
2 {
3     using DbConnection conexion = CrearConexion();
4     conexion.Open();
5
6     using DbCommand comando = conexion.CreateCommand();
7     comando.CommandText = "DELETE FROM Libros WHERE Id = $id";
8     Agregar(comando, "$id", id);
9     return comando.ExecuteNonQuery() == 1;
10 }
```

El número de filas afectadas distingue éxito y ausencia. Una restricción de clave foránea puede impedir el borrado; el repositorio debe traducir ese error a un resultado comprensible para la capa superior.

16. Encapsular un CRUD completo

CSHARP

```
1 public sealed class LibroRepository(string cadenaConexion)
2 {
3     private DbConnection CrearConexion() =>
4         new SqlConnection(cadenaConexion);
5
6     public long Crear(LibroCrear entrada) { /* INSERT parametrizado */ }
7     public Libro? ObtenerPorId(long id) { /* SELECT y reader */ }
8     public List<Libro> ObtenerTodos(int pagina, int tamano) { /* SELECT */ }
9     public bool Actualizar(long id, LibroEditar entrada) { /* UPDATE */ }
10    public bool Eliminar(long id) { /* DELETE */ }
11 }
```

Responsabilidades

Capa	Debe contener	Debe evitar
Repositorio	SQL, parámetros, mapeo y transacción de persistencia	mensajes de interfaz
Servicio	reglas y coordinación de casos de uso	detalles de DbDataReader
Interfaz o endpoint	entrada, salida y presentación	SQL incrustado

No ocultes todo tras un repositorio genérico

SQL directo suele elegirse para controlar consultas concretas. Una abstracción genérica con decenas de opciones puede ser más difícil de comprender y optimizar que repositorios orientados a casos de uso.

17. Transacciones ADO.NET

Una transacción agrupa varias sentencias como una unidad atómica: todas se confirman o ninguna queda aplicada. En SQLite solo una transacción puede mantener cambios pendientes a la vez.

CSHARP

```
1  using DbConnection conexion = CrearConexion();
2  conexion.Open();
3  using DbTransaction transaccion = conexion.BeginTransaction();
4
5  try
6  {
7      using DbCommand prestar = conexion.CreateCommand();
8      prestar.Transaction = transaccion;
9      prestar.CommandText = ""
10         INSERT INTO Prestamos (LibroId, Usuario, Fecha)
11         VALUES ($libroId, $usuario, $fecha)
12         """;
13     Agregar(prestar, "$libroId", libroId);
14     Agregar(prestar, "$usuario", usuario);
15     Agregar(prestar, "$fecha", DateTime.UtcNow);
16     prestar.ExecuteNonQuery();
17
18     using DbCommand marcar = conexion.CreateCommand();
19     marcar.Transaction = transaccion;
20     marcar.CommandText = "UPDATE Libros SET Disponible = 0 WHERE Id = $id";
21     Agregar(marcar, "$id", libroId);
22     marcar.ExecuteNonQuery();
23
24     transaccion.Commit();
25 }
26 catch
27 {
28     transaccion.Rollback();
29     throw;
30 }
```

- Todos los comandos deben usar la misma conexión y asignar la misma Transaction.
- La transacción debe ser breve: no esperes entrada del usuario ni una API remota dentro.
- Savepoints permiten revertir una parte sin cancelar todavía toda la transacción.
- System.Transactions no está implementado por Microsoft.Data.Sqlite; usa transacciones ADO.NET.

18. API asíncrona y la realidad de SQLite

CSHARP

```
1 public async Task<Libro?> ObtenerPorIdAsync(long id, CancellationToken ct)
2 {
3     await using DbConnection conexion = CrearConexion();
4     await conexion.OpenAsync(ct);
5
6     await using DbCommand comando = conexion.CreateCommand();
7     comando.CommandText = """
8         SELECT Id, Titulo, Isbn, Año, AutorId
9         FROM Libros WHERE Id = $id
10        """;
11     Agregar(comando, "$id", id);
12
13     await using DbDataReader lector = await comando.ExecuteReaderAsync(ct);
14     return await lector.ReadAsync(ct) ? MapearLibro(lector) : null;
15 }
```

Limitación esencial

SQLite no soporta E/S asíncrona. En Microsoft.Data.Sqlite, los métodos ADO.NET asíncronos ejecutan el trabajo de forma síncrona. Conocer estas firmas sigue siendo útil para contratos portables, pero no libera un hilo durante el acceso nativo a SQLite.

Qué hacer

- No envuelvas cada consulta en Task.Run dentro de un servidor; desplaza trabajo y puede aumentar la contención.
- Mantén las consultas breves, indexadas y con pocas filas.
- Utiliza WAL para mejorar la convivencia entre lectores y un escritor.
- Propaga CancellationToken en código genérico, pero no prometas cancelación instantánea de una llamada nativa ya iniciada.
- EF Core ofrece métodos ToListAsync y SaveChangesAsync, aunque el proveedor SQLite conserva la limitación de E/S subyacente.

19. Errores, locking y timeout

CSHARP

```
1  try
2  {
3      int filas = comando.ExecuteNonQuery();
4  }
5  catch (SQLiteException ex) when (ex.SqliteErrorCode == 19)
6  {
7      throw new ReglaDatosException(
8          "No se puede guardar porque se incumple una restricción.", ex);
9  }
10 catch (SQLiteException ex) when (ex.SqliteErrorCode is 5 or 6)
11 {
12     throw new DatosOcupadosException(
13         "La base de datos está ocupada. Inténtalo de nuevo.", ex);
14 }
```

SQLiteException expone SqliteErrorCode y SqliteExtendedErrorCode. Conserva la excepción interna para diagnóstico, pero evita mostrar rutas, SQL, datos sensibles o mensajes técnicos directamente al usuario.

CSHARP

```
1  conexion.DefaultTimeout = 60;
2  comando.CommandTimeout = 60;
```

Política de reintento

- Microsoft.Data.Sqlite reintenta automáticamente busy y locked hasta agotar CommandTimeout.
- Un reintento de aplicación debe ser limitado, usar espera creciente y repetir toda la transacción cuando proceda.
- No reintentes errores de sintaxis, restricciones o datos inválidos.
- Un timeout mayor no arregla una transacción innecesariamente larga.
- Registra código extendido, operación y duración sin guardar valores sensibles.

20. Pooling y ciclo de conexiones

El pooling reutiliza recursos internos de conexiones cerradas con la misma cadena. En Microsoft.Data.Sqlite está activado de forma predeterminada. Por eso crear, abrir y desechar una conexión por operación o unidad de trabajo es el patrón correcto.

Práctica	Consecuencia
using breve	la conexión vuelve al pool incluso ante excepción
cadena idéntica	permite reutilizar el mismo grupo
conexión global	riesgo de concurrencia, estado y bloqueo prolongado
Pooling=False	útil solo para diagnóstico o necesidades muy específicas
ClearPool/ClearAllPools	descarta conexiones agrupadas cuando es necesario

CSHARP

```
1 var opciones = new SqliteConnectionStringBuilder
2 {
3     DataSource = ruta,
4     Pooling = true,
5     DefaultTimeout = 30
6 };
```

Objetos no thread-safe

Pooling no convierte una instancia de `SqliteConnection` en segura para varios hilos. Cada operación concurrente debe crear su propia conexión y desecharla correctamente.

21. WAL y concurrencia en SQLite

SQLite bloquea con mayor intensidad que un servidor de bases de datos. Write-ahead logging permite que lectores continúen mientras existe un escritor, aunque sigue habiendo un único escritor efectivo.

CSHARP

```
1 using DbConnection conexion = new SQLiteConnection(cadenaConexion);
2 conexion.Open();
3
4 using DbCommand comando = conexion.CreateCommand();
5 comando.CommandText = "PRAGMA journal_mode = WAL;";
6 string modo = (string)comando.ExecuteScalar() ?? "desconocido";
```

- Configura WAL una vez por base y verifica el modo devuelto.
- No mezcles Cache=Shared y WAL salvo que hayas medido y comprendido el bloqueo; la documentación desaconseja la combinación.
- Mantén transacciones de escritura muy cortas.
- Una aplicación con mucha escritura concurrente puede necesitar un motor cliente-servidor.
- EF Core habilita WAL de forma predeterminada en bases creadas por su proveedor SQLite.

22. Aislamiento, transacciones diferidas y savepoints

Las transacciones SQLite son serializables de forma predeterminada. Microsoft.Data.Sqlite interpreta el nivel solicitado como mínimo y lo promueve a ReadUncommitted o Serializable según las capacidades.

Opción	Comportamiento
Serializable	aislamiento habitual y consistente
ReadUncommitted	solo con caché compartida; permite lecturas sucias
Snapshot y Chaos	no compatibles con Microsoft.Data.Sqlite
Deferred	pospone la transacción real hasta el primer comando
Savepoint	marca un punto al que volver dentro de la transacción

CSHARP

```
1 using SqliteTransaction tx = conexion.BeginTransaction(deferred: true);
2 tx.Save("antes-de-actualizar");
3
4 try
5 {
6     comando.Transaction = tx;
7     comando.ExecuteNonQuery();
8     tx.Release("antes-de-actualizar");
9     tx.Commit();
10 }
11 catch
12 {
13     tx.Rollback("antes-de-actualizar");
14     tx.Rollback();
15     throw;
16 }
```

Actualización de diferida a escritura

Una transacción diferida puede fallar al intentar convertirse en escritura si la base está bloqueada. En ese caso debe repetirse la transacción completa, no solo la última sentencia.

23. CommandType y capacidades del proveedor

ADO.NET define propiedades generales que no todos los motores implementan. Consultar la capacidad del proveedor evita escribir código que compila pero no tiene sentido con SQLite.

Característica	Microsoft.Data.Sqlite
CommandType.Text	compatible; es el modo normal
CommandType.StoredProcedure	no compatible: SQLite no tiene procedimientos almacenados
Parámetros con nombre	compatibles con \$, @ o :
Parámetros posicionales	no compatibles
Parámetros de salida	no compatibles
DbConnection.GetSchema	no implementado
GetSchemaTable del reader	compatible para metadatos del resultado
System.Transactions	no compatible; usar DbTransaction

CSHARP

```
1 using DbCommand comando = conexion.CreateCommand();
2 comando.CommandType = CommandType.Text;
3 comando.CommandText = "SELECT Id, Titulo FROM Libros";
```

24. DataTable y DataSet

DataTable representa una tabla desconectada en memoria con columnas, filas, restricciones y expresiones. Puede resultar útil para enlazado de controles, interoperabilidad con API antiguas, informes o transformaciones tabulares.

CSHARP

```
1 using DbCommand comando = conexion.CreateCommand();
2 comando.CommandText = """
3     SELECT l.Id, l.Titulo, a.Nombre AS Autor
4     FROM Libros l
5     JOIN Autores a ON a.Id = l.AutorId
6     ORDER BY l.Titulo
7     """;
8
9 using DbDataReader lector = comando.ExecuteReader();
10 var tabla = new DataTable("Libros");
11 tabla.Load(lector);
12
13 foreach (DataRow fila in tabla.Rows)
14 {
15     Console.WriteLine($"{fila["Titulo"]} - {fila["Autor"]}");
16 }
```

Límite del proveedor SQLite

Microsoft.Data.Sqlite no implementa DbDataAdapter. DataTable y DataSet pueden **cargar** datos desde un reader, pero no actualizar automáticamente la base mediante un adaptador. Las altas, modificaciones y bajas se realizan con comandos explícitos.

DbDataReader	DataTable
conectado y secuencial	desconectado y completamente en memoria
poco consumo	consume memoria proporcional al resultado
ideal para mapear o transmitir	útil para tablas dinámicas e integración antigua
tipado mediante accedores	columnas y valores como modelo tabular

25. Metadatos, PRAGMA y esquema

Como `DbConnection.GetSchema` no está implementado, el esquema se consulta mediante las API propias de SQLite: `sqlite_schema` y sentencias `PRAGMA`.

SQL

```
1 SELECT name, type, sql
2 FROM sqlite_schema
3 WHERE type IN ('table', 'index', 'view')
4 ORDER BY type, name;
5
6 PRAGMA table_info('Libros');
7 PRAGMA foreign_key_list('Libros');
8 PRAGMA index_list('Libros');
9 PRAGMA journal_mode;
10 PRAGMA foreign_keys;
```

CSHARP

```
1 using DbCommand comando = conexion.CreateCommand();
2 comando.CommandText = "PRAGMA table_info('Libros')";
3 using DbDataReader lector = comando.ExecuteReader();
4
5 while (lector.Read())
6 {
7     Console.WriteLine($"{lector["name"]}: {lector["type"]}");
8 }
```

No paramétrices identificadores

El nombre de una tabla en `PRAGMA` o `SQL` no puede sustituirse con un parámetro normal. Si debe ser dinámico, compáralo con una lista cerrada de identificadores permitidos.

26. Rendimiento con SQL directo

Índices y plan

SQL

```
1 CREATE INDEX IF NOT EXISTS IX_Libros_Titulo_Anio
2 ON Libros(Titulo, Anio);
3
4 EXPLAIN QUERY PLAN
5 SELECT Id, Titulo
6 FROM Libros
7 WHERE Titulo = $titulo AND Anio >= $anio;
```

- Selecciona solo las columnas necesarias; evita SELECT * en contratos estables.
- Parametriza valores y reutiliza la forma de las consultas.
- Agrupa escrituras masivas en una transacción; confirma una vez.
- Indexa claves foráneas, búsquedas y órdenes frecuentes, pero evita índices redundantes.
- Mide con datos representativos y EXPLAIN QUERY PLAN.
- Limita resultados y usa lectura secuencial para BLOB grandes.

BLOB incremental

SqliteBlob permite transmitir un BLOB sin cargarlo entero. Se reserva el tamaño con zeroblob, se obtiene el rowid y se escribe o lee mediante Stream. Es una técnica avanzada para contenidos grandes; para archivos normales también puede ser mejor almacenar una ruta y gestionar el fichero fuera de la base.

27. Probar código ADO.NET

Una base en memoria ofrece aislamiento y velocidad, pero solo existe mientras permanece abierta al menos una conexión. Debe ejecutarse el mismo esquema que en producción.

CSHARP

```
1 public sealed class BaseSqlitePruebas : IDisposable
2 {
3     public SqliteConnection Conexion { get; } =
4         new("Data Source=:memory:");
5
6     public BaseSqlitePruebas()
7     {
8         Conexion.Open();
9         using var comando = Conexion.CreateCommand();
10        comando.CommandText = File.ReadAllText("schema.sql");
11        comando.ExecuteNonQuery();
12    }
13
14    public void Dispose() => Conexion.Dispose();
15 }
```

Qué comprobar

- Crear devuelve un identificador y persiste todos los campos.
- Buscar devuelve null cuando el id no existe.
- Actualizar y eliminar distinguen una fila de cero filas.
- Los parámetros tratan comillas y texto parecido a SQL como datos.
- UNIQUE, CHECK y claves foráneas rechazan estados inválidos.
- Una excepción intermedia revierte toda la transacción.
- Dos conexiones reproducen el comportamiento de locking relevante.

28. Qué es un ORM y qué problema resuelve

Un ORM relaciona objetos de la aplicación con tablas, filas y columnas. Entity Framework Core mantiene un modelo, traduce LINQ a SQL, materializa entidades, sigue cambios y genera comandos de persistencia.

Trabajo	ADO.NET	EF Core
SQL	escrito directamente	generado desde LINQ o proporcionado cuando hace falta
Mapeo	manual desde DbDataReader	materialización automática
Cambios	UPDATE explícito	change tracker y SaveChanges
Relaciones	JOIN y agrupación manual	navegaciones y Include
Esquema	scripts o sistema propio	migraciones
Control	máximo y visible	alto, con una capa de abstracción

Qué no resuelve

- No diseña por sí solo un buen modelo relacional.
- No elimina la necesidad de índices, restricciones, transacciones ni conocer SQL.
- No garantiza que una consulta LINQ sea eficiente.
- No sustituye reglas de negocio ni validación de entrada.
- No oculta las limitaciones del proveedor SQLite.

29. Instalar y configurar EF Core 10 con SQLite

Terminal

```
1 dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version 10.0.0
2 dotnet add package Microsoft.EntityFrameworkCore.Design --version 10.0.0
3 dotnet tool install --global dotnet-ef --version 10.0.0
4 dotnet ef --version
```

Aplicación de consola

CSHARP

```
1 var opciones = new DbContextOptionsBuilder<BibliotecaContext>()
2     .UseSqlite(cadenaConexion)
3     .EnableDetailedErrors()
4     .Options;
5
6 await using var db = new BibliotecaContext(opciones);
```

ASP.NET Core

Program.cs

```
1 builder.Services.AddDbContext<BibliotecaContext>(options =>
2     options.UseSqlite(
3         builder.Configuration.GetConnectionString("Biblioteca"));
```

Versiones alineadas

Utiliza paquetes Microsoft.EntityFrameworkCore.* de la misma línea 10.0.x. El proveedor incluye las dependencias base y se apoya en Microsoft.Data.Sqlite.

30. Crear entidades

Models/Entidades.cs

```
1 public sealed class Autor
2 {
3     public long Id { get; set; }
4     public required string Nombre { get; set; }
5     public string? Pais { get; set; }
6     public List<Libro> Libros { get; set; } = [];
7 }
8
9 public sealed class Libro
10 {
11     public long Id { get; set; }
12     public required string Titulo { get; set; }
13     public required string Isbn { get; set; }
14     public int Anio { get; set; }
15     public bool Disponible { get; set; } = true;
16
17     public long AutorId { get; set; }
18     public Autor Autor { get; set; } = null!;
19 }
```

Por convención, Id es clave primaria; AutorId es clave foránea y Autor/Libros son navegaciones. **required** obliga al código C# a inicializar la propiedad, pero la nulabilidad y las restricciones del modelo siguen configurándose en EF Core y SQLite.

Elemento	Significado
Entidad	tipo que EF incluye en el modelo
Propiedad escalar	valor que suele mapearse a una columna
Clave primaria	identidad de la fila
Clave foránea	valor que referencia la entidad principal
Navegación	referencia o colección para recorrer una relación

31. DbContext y configuración Fluent

C#

```
1 public sealed class BibliotecaContext(  
2     DbContextOptions<BibliotecaContext> options) : DbContext(options)  
3 {  
4     public DbSet<Autor> Autores => Set<Autor>();  
5     public DbSet<Libro> Libros => Set<Libro>();  
6  
7     protected override void OnModelCreating(ModelBuilder modelBuilder)  
8     {  
9         modelBuilder.Entity<Autor>(entidad =>  
10             {  
11                 entidad.Property(x => x.Nombre).HasMaxLength(120).IsRequired();  
12             });  
13  
14         modelBuilder.Entity<Libro>(entidad =>  
15             {  
16                 entidad.Property(x => x.Titulo).HasMaxLength(160).IsRequired();  
17                 entidad.Property(x => x.Isbn).HasMaxLength(13).IsRequired();  
18                 entidad.HasIndex(x => x.Isbn).IsUnique();  
19                 entidad.ToTable(t => t.HasCheckConstraint(  
20                     "CK_Libros_Anio", "Anio BETWEEN 1450 AND 2100"));  
21             });  
22     }  
23 }
```

Convenciones, atributos y Fluent API

- Las convenciones reducen configuración para patrones habituales.
- DataAnnotations coloca reglas sencillas junto al modelo.
- Fluent API permite relaciones, índices, conversiones y detalles sin acoplar la clase.
- La configuración Fluent prevalece sobre atributos y convenciones.
- Divide configuraciones grandes mediante IEntityTypeConfiguration.

32. Migraciones: evolucionar el esquema

Las migraciones comparan el modelo actual con el snapshot anterior, generan operaciones Up y Down y registran las aplicadas en `__EFMigrationsHistory`.

```
BASH
1 dotnet ef migrations add Inicial
2 dotnet ef database update
3 dotnet ef migrations list
4
5 # Después de añadir una propiedad o relación
6 dotnet ef migrations add AgregarPrestamos
7 dotnet ef database update
8
9 # Script revisable para despliegue
10 dotnet ef migrations script --idempotent
```

Buenas prácticas

1. Nombra la migración como el cambio: AgregarPrestamos, no Cambio2.
2. Lee el código y el SQL generado antes de aplicarlos.
3. Incluye migraciones y snapshot en el control de versiones.
4. No elimines una migración ya aplicada en entornos compartidos.
5. Prueba actualización sobre una copia con datos reales.
6. Para producción, prefiere scripts revisados o bundles frente a migrar al arrancar cada réplica.

EnsureCreated no es Migrations

EnsureCreated crea un esquema sin historial y es útil para pruebas o prototipos descartables. No lo llares antes de Migrate: ambos enfoques no deben mezclarse para la misma base.

33. CRUD EF Core: crear y consultar

Crear

CSHARP

```
1 var autor = new Autor { Nombre = "Carmen Laforet", Pais = "España" };
2 var libro = new Libro
3 {
4     Titulo = "Nada",
5     Isbn = "9788423342368",
6     Anio = 1945,
7     Autor = autor
8 };
9
10 db.Libros.Add(libro);
11 await db.SaveChangesAsync();
12 Console.WriteLine($"Nuevo id: {libro.Id}");
```

Consultar

CSHARP

```
1 Libro? detalle = await db.Libros
2     .AsNoTracking()
3     .Include(x => x.Autor)
4     .SingleOrDefaultAsync(x => x.Id == id);
5
6 List<LibroResumen> listado = await db.Libros
7     .AsNoTracking()
8     .OrderBy(x => x.Titulo)
9     .Select(x => new LibroResumen(
10         x.Id, x.Titulo, x.Autor.Nombre, x.Anio))
11     .Take(100)
12     .ToListAsync();
```

La proyección crea DTO directamente en SQL y evita materializar columnas o entidades completas. AsNoTracking reduce trabajo en consultas que no se van a modificar.

34. CRUD EF Core: actualizar y eliminar

Actualizar entidad seguida

CSHARP

```
1  Libro? libro = await db.Libros.FindAsync(id);
2  if (libro is null) return false;
3
4  libro.Titulo = entrada.Titulo;
5  libro.Anio = entrada.Anio;
6  await db.SaveChangesAsync();
7  return true;
```

Eliminar entidad seguida

CSHARP

```
1  Libro? libro = await db.Libros.FindAsync(id);
2  if (libro is null) return false;
3
4  db.Libros.Remove(libro);
5  await db.SaveChangesAsync();
6  return true;
```

Operaciones directas

CSHARP

```
1  int actualizados = await db.Libros
2      .Where(x => x.AutorId == autorId)
3      .ExecuteUpdateAsync(cambios => cambios
4          .SetProperty(x => x.Disponible, false));
5
6  int eliminados = await db.Libros
7      .Where(x => x.Id == id)
8      .ExecuteDeleteAsync();
```

ExecuteUpdate y ExecuteDelete ejecutan SQL inmediato sin cargar ni seguir entidades. No esperan a SaveChanges y no aplican automáticamente la concurrencia del change tracker.

35. Seguimiento de cambios y estados

Estado	Significado al guardar
Detached	EF no sigue la instancia
Unchanged	existe y no se detectan cambios
Added	generará INSERT
Modified	generará UPDATE para propiedades modificadas
Deleted	generará DELETE

CSHARP

```
1 EntityEntry<Libro> entrada = db.Entry(libro);
2 Console.WriteLine(entrada.State);
3
4 libro.Titulo = "Nuevo título";
5 db.ChangeTracker.DetectChanges();
6 Console.WriteLine(entrada.State);
7
8 await db.SaveChangesAsync();
9 Console.WriteLine(entrada.State);
```

- DbContext está pensado como una unidad de trabajo corta.
- No es thread-safe ni debe ejecutar operaciones paralelas sobre la misma instancia.
- SaveChanges detecta cambios y acepta los estados tras guardar.
- AsNoTracking es adecuado para lectura; no modifiques esperando que SaveChanges lo detecte.
- Clear elimina todas las entidades seguidas cuando una operación larga lo necesita.

36. LINQ y traducción a SQL

CSHARP

```
1 IQueryable<Libro> consulta = db.Libros.AsNoTracking();
2
3 if (!string.IsNullOrWhiteSpace(texto))
4     consulta = consulta.Where(x => x.Titulo.Contains(texto));
5
6 if (desde is not null)
7     consulta = consulta.Where(x => x.Anio >= desde);
8
9 List<LibroResumen> resultado = await consulta
10     .OrderBy(x => x.Titulo)
11     .ThenBy(x => x.Id)
12     .Skip((pagina - 1) * tamano)
13     .Take(tamano)
14     .Select(x => new LibroResumen(x.Id, x.Titulo, x.Autor.Nombre, x.Anio))
15     .ToListAsync();
```

IQueryable frente a IEnumerable

Mientras la consulta es IQueryable, EF construye una expresión y trata de traducirla a SQL. ToListAsync, SingleAsync o la enumeración ejecutan la consulta. Convertir pronto a IEnumerable o ToList mueve filtros posteriores a memoria.

Inspección

CSHARP

```
1 string sql = consulta.ToQueryString();
2 Console.WriteLine(sql);
```

No todo C# se traduce

Utiliza operaciones traducibles y prueba las consultas. Las comparaciones u ordenaciones de ciertos tipos SQLite, como decimal o DateTimeOffset, pueden requerir evaluación cliente o un mapeo diferente.

37. Relación uno a muchos

CSHARP

```
1 modelBuilder.Entity<Libro>()
2     .HasOne(x => x.Autor)
3     .WithMany(x => x.Libros)
4     .HasForeignKey(x => x.AutorId)
5     .OnDelete(DeleteBehavior.Restrict);
```

Pieza	En Autor - Libro
Principal	Autor
Dependiente	Libro
Clave principal	Autor.Id
Clave foránea	Libro.AutorId
Navegación de referencia	Libro.Autor
Navegación de colección	Autor.Libros

Una relación requerida usa una clave foránea no nullable. En una relación opcional, `AutorId` sería `long?` y la navegación `Autor` podría ser nullable. La política de borrado debe expresar el dominio: `Cascade`, `Restrict`, `SetNull` o `ClientCascade`, teniendo en cuenta las capacidades del proveedor.

38. Uno a uno y muchos a muchos

Uno a uno

CSHARP

```
1 modelBuilder.Entity<Autor>()
2     .HasOne(x => x.Perfil)
3     .WithOne(x => x.Autor)
4     .HasForeignKey<PerfilAutor>(x => x.AutorId);
```

En uno a uno, la clave foránea única se sitúa en el dependiente. Puede compartir la clave primaria con la entidad principal.

Muchos a muchos

CSHARP

```
1 public sealed class Etiqueta
2 {
3     public long Id { get; set; }
4     public required string Nombre { get; set; }
5     public List<Libro> Libros { get; set; } = [];
6 }
7
8 modelBuilder.Entity<Libro>()
9     .HasMany(x => x.Etiquetas)
10    .WithMany(x => x.Libros)
11    .UsingEntity("LibroEtiqueta");
```

EF Core puede gestionar la tabla puente sin una clase explícita. Si la relación tiene datos propios, como FechaAsignacion u Orden, crea una entidad LibroEtiqueta con dos relaciones uno a muchos.

39. Cargar relaciones

Estrategia	Cómo	Cuándo
Eager	Include y ThenInclude	se conoce el grafo necesario
Explícita	Entry(...).Reference/Collection.Load	se decide después de materializar
Lazy	proxies o ILazyLoader	acceso transparente con riesgo de N+1
Proyección	Select a DTO	API y lecturas eficientes

CSHARP

```
1 Autor? autor = await db.Autores
2     .Include(x => x.Libros.OrderBy(l => l.Titulo))
3     .SingleOrDefaultAsync(x => x.Id == autorId);
4
5 var dto = await db.Autores
6     .Where(x => x.Id == autorId)
7     .Select(x => new
8     {
9         x.Id,
10        x.Nombre,
11        Libros = x.Libros.Select(l => new { l.Id, l.Titulo })
12    })
13     .SingleOrDefaultAsync();
```

Evita N+1

Cargar una colección dentro de un bucle puede ejecutar una consulta por fila. Proyecta o incluye las relaciones necesarias y observa el SQL. Para varios Include de colecciones, considera AsSplitQuery tras medir.

40. Transacciones y concurrencia en EF Core

Una llamada a `SaveChanges` es transaccional cuando el proveedor lo permite. Una transacción explícita se reserva para varias llamadas o una coordinación que deba ser atómica.

CSHARP

```
1  await using var tx = await db.Database.BeginTransactionAsync();
2  try
3  {
4      prestamo.FechaDevolucion = DateTime.UtcNow;
5      await db.SaveChangesAsync();
6
7      libro.Disponible = true;
8      await db.SaveChangesAsync();
9
10     await tx.CommitAsync();
11 }
12 catch
13 {
14     await tx.RollbackAsync();
15     throw;
16 }
```

Token de concurrencia gestionado por la aplicación

CSHARP

```
1  public sealed class Libro
2  {
3      public long Id { get; set; }
4      [ConcurrencyCheck]
5      public int Version { get; set; }
6  }
7
8  libro.Version++;
9  try
10 {
11     await db.SaveChangesAsync();
12 }
13 catch (DbUpdateConcurrencyException)
14 {
15     // Recargar, informar o resolver el conflicto.
16 }
```

SQLite no ofrece un rowversion autogenerado. Puede usarse un entero, Guid u otro token actualizado por la aplicación. EF incorpora el valor original en el WHERE y detecta cero filas afectadas.

41. SQL directo e interoperabilidad con EF Core

Consultas SQL parametrizadas

CSHARP

```
1 string prefijo = "Na";
2
3 List<Libro> libros = await db.Libros
4     .FromSqlInterpolated($"
5         SELECT * FROM Libros
6         WHERE Titulo LIKE {prefijo + "%"}
7     ")
8     .AsNoTracking()
9     .ToListAsync();
10
11 int filas = await db.Database.ExecuteSqlInterpolatedAsync($"
12     UPDATE Libros SET Disponible = {false}
13     WHERE AutorId = {autorId}
14 ");
```

Las variantes interpoladas convierten valores en parámetros. Raw exige controlar parámetros manualmente. Los identificadores dinámicos siguen necesitando una lista permitida.

Compartir conexión y transacción

CSHARP

```
1 DbConnection conexion = db.Database.GetDbConnection();
2 await db.Database.OpenConnectionAsync();
3
4 try
5 {
6     await using DbCommand comando = conexion.CreateCommand();
7     comando.Transaction = db.Database.CurrentTransaction?.GetDbTransaction();
8     comando.CommandText = "SELECT COUNT(*) FROM Libros";
9     long total = (long)(await comando.ExecuteScalarAsync() ?? 0L);
10 }
11 finally
12 {
13     await db.Database.CloseConnectionAsync();
14 }
```

Propiedad de la conexión

Si EF Core abrió la conexión, deja que EF la cierre. Si la abres explícitamente para interoperar, ciérrala en finally y no ejecute operaciones paralelas sobre el mismo DbContext.

42. Limitaciones de EF Core con SQLite

Área	Limitación o precaución
Esquemas	SQLite no los soporta; EnsureSchema no hace nada
Secuencias	no compatibles
Rowversion	no hay token autogenerado; usar token de aplicación
decimal	comparar u ordenar puede requerir cliente; valorar double o conversión
DateTimeOffset	preferir DateTime UTC y convertir en los bordes
TimeSpan y ulong	operaciones distintas de igualdad pueden no traducirse
Migraciones	muchos cambios requieren reconstruir la tabla
Artefactos manuales	una reconstrucción no puede preservar lo que no forma parte del modelo

El proveedor intenta reconstruir tablas para operaciones como AlterColumn, añadir claves o restricciones y algunos borrados. Revisa siempre la migración y conserva copias de seguridad antes de cambios destructivos.

SQLite no es SQL Server pequeño

El modelo relacional es parecido, pero tipos, concurrencia, ALTER TABLE y capacidades operativas difieren. Diseña para el motor que realmente vas a usar.

43. Rendimiento con EF Core

- Proyecta a DTO y selecciona únicamente columnas necesarias.
- Usa AsNoTracking en lectura sin actualización.
- Limita, ordena y pagina en SQL antes de materializar.
- Evita N+1 y revisa Include de varias colecciones.
- Examina ToQueryString y logs de Microsoft.EntityFrameworkCore.Database.Command.
- Crea índices desde el modelo y confirma que las consultas los utilizan.
- Usa ExecuteUpdate y ExecuteDelete para cambios masivos sin seguimiento.
- No compartas DbContext entre operaciones concurrentes.

CSHARP

```
1 options.LogTo(  
2     Console.WriteLine,  
3     new[] { DbLoggerCategory.Database.Command.Name },  
4     LogLevel.Information);  
5  
6 string sql = db.Libros  
7     .Where(x => x.Anio >= 2000)  
8     .Select(x => new { x.Id, x.Titulo })  
9     .ToQueryString();
```

Consultas compiladas

EF.CompileQuery y EF.CompileAsyncQuery reducen el coste de compilar expresiones muy repetidas. Solo deben incorporarse después de medir; el tiempo de SQLite o la cantidad de datos suele dominar antes que la compilación LINQ.

44. Probar EF Core con SQLite

CSHARP

```
1 var conexion = new SQLiteConnection("Data Source=:memory:");
2 await conexion.OpenAsync();
3
4 var opciones = new DbContextOptionsBuilder<BibliotecaContext>()
5     .UseSqlite(conexion)
6     .Options;
7
8 await using (var preparacion = new BibliotecaContext(opciones))
9 {
10     await preparacion.Database.EnsureCreatedAsync();
11     preparacion.Autores.Add(new Autor { Nombre = "Autora de prueba" });
12     await preparacion.SaveChangesAsync();
13 }
14
15 await using (var prueba = new BibliotecaContext(opciones))
16 {
17     int total = await prueba.Autores.CountAsync();
18     Assert.Equal(1, total);
19 }
20
21 await conexion.DisposeAsync();
```

Mantener la conexión abierta conserva la base en memoria entre contextos. SQLite en memoria prueba SQL relacional, restricciones y traducción mejor que el proveedor EF InMemory, cuyo comportamiento no pretende imitar una base relacional.

Matriz mínima

- CRUD de éxito y ausencia.
- Restricciones únicas, nulos y claves foráneas.
- Relaciones y políticas de borrado.
- Traducción de filtros, orden y paginación.
- Transacción y rollback.
- Conflicto de concurrencia.
- Aplicación de migraciones sobre una base temporal cuando se prueban migraciones.

45. Elegir ADO.NET, EF Core o ambos

Criterio	ADO.NET	EF Core
Control de SQL	total y explícito	alto, con traducción y SQL directo opcional
Velocidad de desarrollo	más código repetitivo	alta para dominios y CRUD
Mapeo	manual	automático y configurable
Relaciones	JOIN y ensamblado manual	navigaciones y carga
Migraciones	no incluidas	incluidas
Seguimiento	no existe	change tracker opcional
Consulta excepcional	muy natural	LINQ o SQL directo

Criterio práctico

1. Empieza por claridad y necesidades del proyecto, no por una guerra de tecnologías.
2. Usa EF Core para la mayoría del dominio si reduce mapeo y mantenimiento.
3. Usa ADO.NET cuando una consulta concreta necesita control, streaming o un contrato tabular.
4. Mide antes de sustituir EF Core por SQL directo por rendimiento.
5. Si combinas ambos, comparte transacción de manera explícita y documenta quién gestiona el esquema.

Conocimiento transferible

Aprender ADO.NET ayuda a diagnosticar EF Core: detrás de LINQ siguen existiendo una conexión, un comando, parámetros, un lector y, a menudo, una transacción.

46. Proyecto integrador: Biblioteca de aula

Parte A: ADO.NET

1. Crea el esquema de autores, libros, etiquetas y préstamos con restricciones e índices.
2. Construye la cadena mediante SqlConnectionStringBuilder.
3. Implementa CRUD de libros con DbConnection, DbCommand y parámetros.
4. Mapea listados y detalle mediante DbDataReader.
5. Implementa préstamo y devolución como transacciones.
6. Añade búsqueda, paginación, timeout y traducción segura de SqliteException.
7. Carga un informe de libros y autores en DataTable.

Parte B: EF Core

1. Crea entidades, DbContext y configuraciones Fluent equivalentes.
2. Genera la migración Inicial y revisa el SQL.
3. Implementa CRUD y proyecciones a DTO.
4. Añade relaciones uno a muchos y muchos a muchos.
5. Prueba carga mediante Include, proyección y consulta explícita.
6. Añade un token de concurrencia gestionado por la aplicación.
7. Compara SQL, cantidad de código y comportamiento de ambos enfoques.

Entrega

Evidencia	Criterio
Código	legible, parametrizado, sin conexiones ni contextos globales
Base	restricciones e índices verificables
Migraciones	versionadas, revisadas y reproducibles
Pruebas	CRUD, relaciones, error, rollback y concurrencia
Informe	decisión razonada ADO.NET frente a EF Core

47. Diagnóstico de errores frecuentes

Síntoma	Causa probable	Comprobación
No such table	ruta a otra base o esquema sin crear	muestra ruta absoluta y sqlite_schema
Database is locked	transacción larga o conexión compartida	revisa duración, WAL y timeout
Foreign key no actúa	PRAGMA desactivado	consulta PRAGMA foreign_keys
Parámetro no encontrado	nombre distinto	compara prefijo, texto y ParameterName
Invalid cast	tipo SQLite inesperado	usa GetFieldType y accesoros correctos
Object disposed	lector o conexión fuera de alcance	revisa using y materialización
EF no guarda cambios	entidad no seguida o sin cambios	inspecciona Entry.State
N+1	carga dentro de un bucle	activa log SQL y proyecta
Migración falla	operación SQLite no soportada	revisa reconstrucción y artefactos manuales
Async no mejora	SQLite hace E/S síncrona	optimiza SQL, índices y concurrencia

Secuencia de diagnóstico

1. Confirma la ruta absoluta y la cadena efectiva sin mostrar claves.
2. Comprueba que la conexión abre y qué fichero está utilizando.
3. Ejecuta el SQL con parámetros representativos y revisa el código de error.
4. Inspecciona esquema, foreign_keys, journal_mode e índices.
5. En EF Core, muestra ToQueryString y estados del ChangeTracker.
6. Añade una prueba que reproduzca el fallo antes de corregirlo.

48. Checklist final

ADO.NET

- Conexiones, comandos, lectores y transacciones se desechan con using.
- Todos los valores externos se parametrizan.
- Se distingue ExecuteNonQuery, ExecuteScalar y ExecuteReader.
- Los nulos usan DBNull.Value e IsDBNull.
- El CRUD comprueba filas afectadas y restricciones.
- Las transacciones son breves y todos sus comandos las referencian.
- Timeout, locking, pooling y ausencia de E/S asíncrona real están documentados.

EF Core

- DbContext tiene un ciclo corto y no se comparte entre hilos.
- Entidades, claves, índices, relaciones y borrados están configurados.
- Las migraciones se revisan, prueban y versionan.
- Las lecturas proyectan y usan AsNoTracking cuando corresponde.
- No existen cargas N+1 ni materializaciones prematuras evidentes.
- Las limitaciones de tipos y migraciones de SQLite se han considerado.
- Las pruebas utilizan SQLite, no una imitación no relacional.

49. Glosario

Término	Definición
ADO.NET	abstracciones .NET para conexión, comando, lectura y transacción
Proveedor	implementación ADO.NET para un motor concreto
Pooling	reutilización de recursos internos de conexiones cerradas
DbDataReader	lector conectado y secuencial de resultados
DataTable	tabla desconectada en memoria
Transacción	unidad de trabajo atómica
Aislamiento	visibilidad de cambios concurrentes
WAL	registro anticipado que mejora convivencia de lectores y escritor
ORM	mapeo entre modelo de objetos y modelo relacional
DbContext	unidad de trabajo y acceso al modelo de EF Core
Migración	cambio versionado del esquema
Change tracker	componente que mantiene estados y cambios de entidades
Navegación	propiedad que recorre una relación
Proyección	selección directa de la forma necesaria del resultado

50. Referencias

[Microsoft.Data.Sqlite: introducción](#)

[Cadenas de conexión SQLite](#)

[Parámetros SQLite](#)

[Tipos de datos SQLite](#)

[Transacciones SQLite](#)

[Errores, locking y timeout](#)

[Limitaciones asíncronas](#)

[Limitaciones ADO.NET del proveedor](#)

[ADO.NET: DataReader](#)

[ADO.NET: DataTable](#)

[Proveedor SQLite para EF Core](#)

[Limitaciones de EF Core con SQLite](#)

[Introducción a EF Core](#)

[DbContext](#)

[Seguimiento de cambios](#)

[Consultas](#)

[Relaciones](#)

[Migraciones](#)

[Guardar datos](#)

[Concurrencia](#)

[Documentación de SQLite](#)

Revisión de versiones

Los ejemplos están orientados a C# 14, .NET 10, Microsoft.Data.Sqlite 10 y EF Core 10. Mantén alineadas las versiones de los paquetes y revisa notas de versión y avisos de seguridad antes de una nueva edición del curso.